Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 12

Roll No : 324

Name : Aditya Chauhan

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

In [ ]:
```
try:
    num1 = int(input("Enter Number"))
```

```
    num2 = int(input("Enter Number"))
    ans = num1//num2
except ZeroDivisionError as err:
    print(err)
except ValueError as err:
    print(err)
except TypeError as err:
    print(err)
```

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

In [16]:
```
l1 = [1,2,3,4]
d1 = {1:"ONE",2:"TWO"}

try:
    ind = int(input("Enter Index: "))
    print("List Value:", l1[ind])

    key1 = int(input("Enter Key: "))
    print("Dictionary Value:", d1[key1])

except IndexError as err:
    print(err)

except KeyError as err:
    print("Not valid Key")
```

```
List Value: 2
Not valid Key
```

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

In [26]:
```
try:
    import abcModule
    fp = open("abc.txt")
    print(fp.read())
except FileNotFoundError as err:
    print(err)
except ModuleNotFoundError as err:
    print(err)
```

```
No module named 'abcModule'
```

04) WAP that handles all type of exceptions in a single except block in standard error message format.

In [28]:
```
try:
    ans = 10//5
    temp = "abc" + 10
except Exception as err:
    print(err)
```

```
can only concatenate str (not "int") to str
```

05) WAP to demonstrate else and finally block.

In [30]:
```
try:
    num1 = int(input("Enter Number"))
    num2 = int(input("Enter Number"))
    ans = num1//num2
except Exception as err:
    print(err)
else:
    print("ans = ",ans)
finally:
    print("Code Complete")
```

```
integer division or modulo by zero
Code Complete
```

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

In [33]:
```
def divide(a,b):
    try:
        return a//b
    except Exception as err:
        print(err)
    finally:
        print("Code Complete")
num1 = int(input("Enter Number"))
num2 = int(input("Enter Number"))
ans = divide(num1,num2)
print(ans)
```

```
integer division or modulo by zero
Code Complete
None
```

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

In [38]:
```
try:
    l1 = list(map(int,input("Enter Prices").split()))
    for i in l1:
        if i<0:
            raise ValueError("Negative Not Valid")
except ValueError as err:
    print(err)
else:
    total = 0
    for i in l1:
       total+=i
    print(total)
```

```
invalid literal for int() with base 10: 'abc'
```

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

In [54]:
```
try:
    s = input("Enter Grades by commas").split(",")
    l1 = [int(i.strip()) for i in s]
except ValueError as err:
    print(err)
else:
    print(l1)
```

```
invalid literal for int() with base 10: 'abc'
```

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

In [44]:
```
l1 = []
try:
    for i in range(5):
        mark = int(input("Enter Marks"))
        assert mark>=0 and mark<=100
        l1.append(mark)
except AssertionError as err:
    print("Invalid Marks")
else:
    ans = 0
    for i in l1:
```

```
        ans+=i
    print(ans//5)
```

```
Invalid Marks
```

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the messsage "Correct" otherwise.

In [50]:
```
class WeakPasswordError(Exception):
    pass
try:
    password = input("Enter password:")
    if len(password) < 8:
        raise WeakPasswordError("Enter Valid Password")
    count = 0
    for i in password:
        if i.isdigit():
            count+=1
    if count<1:
        raise WeakPasswordError("Enter Digit in password")
except WeakPasswordError as err:
    print(err)
else:
    print("Correct")
```

```
Enter Valid Password
```

In [ ]:

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

In [52]:
```
class NegativeNumberError(Exception):
    pass
try:
    digit = int(input("Enter Number:"))
    if digit < 0:
        raise NegativeNumberError("Cannot calculate the square root of a negative number")
except NegativeNumberError as err:
    print(err)
else:
    print(digit**0.5)
```

```
Cannot calculate the square root of a negative number
```

In [ ]: